

Lecture 07a.

Trees : Motivations, Terminology, and Representation

Background Motivations (1)

“Abstract Data Types” (ADT)

- https://en.wikipedia.org/wiki/Abstract_data_type
- These define the behaviour of a data storage system from the perspective of the USER of the storage
- Specifies operations that must be supported, e.g.
 - Find, Insert, Delete, ...
 - But not how they are implemented
- Similar to the public interface of classes in OO
- In contrast the actual implementation might be called “concrete”

“Concrete Data Types” (CDT)

- These define the behaviour of a data storage system from the perspective of the IMPLEMENTATION of the storage
- Similar to the private implementation of classes in OO

Background Motivations (2)

“Abstract Data Types” (ADT)

- Have to implement specified operations
- Will often have some complexity requirements on the operations written in Big-Oh notation
 - E.g. “insertion must be $O(n)$ ”

“Concrete Data Types” (CDT)

- There might often be many potential implementations
- “Inefficient but easy to implement”
 - Often can just use arrays/lists for first implementation
- “Efficient but difficult to implement”
 - Can switch to this later without the users having to change their code

In data structures we can often start with arrays/lists but the more efficient versions tend to be with trees. Often see a reduction from $O(n)$ to $O(\log n)$

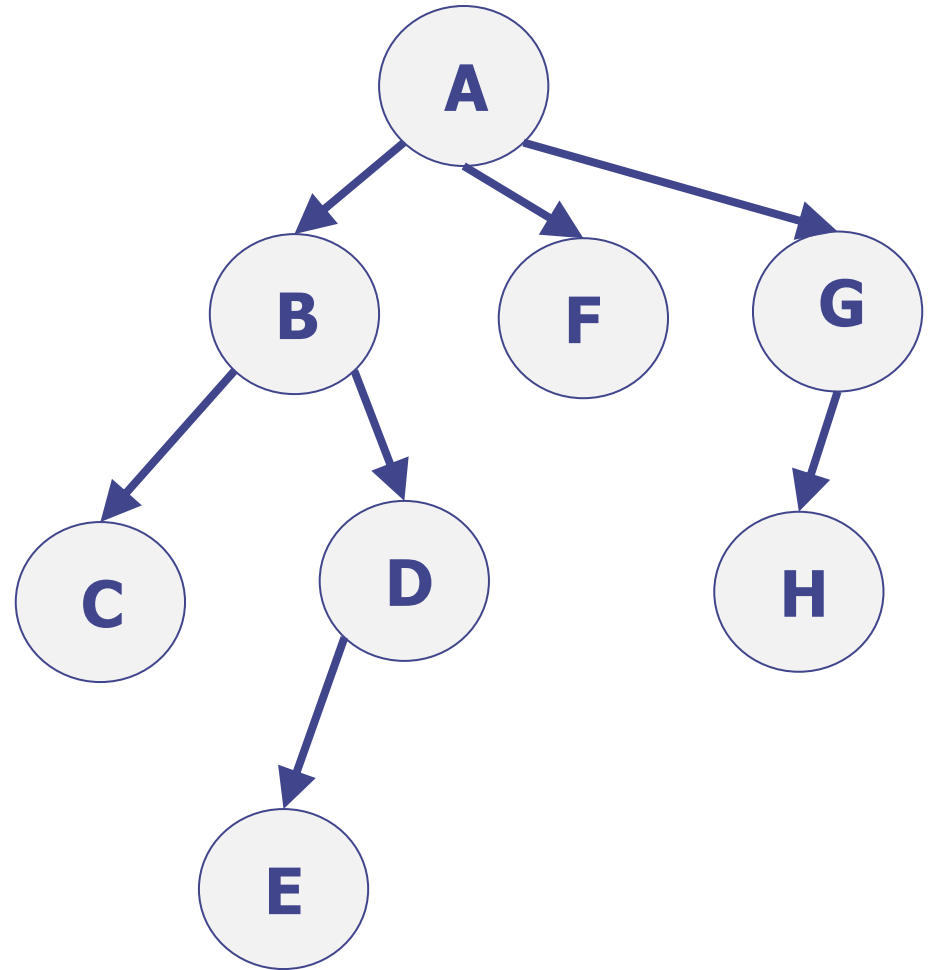
Background Motivations (3)

- Specifications of “library functions” are the ADT, and give some limits on the runtimes – specified in Big-Oh notation
- An ADT might be implemented using different choices for the CDT
 - The choice of CDT will not be apparent from the interface: “data hiding” “encapsulation” – e.g. see ‘Object Oriented Methods’
 - The choice of CDT will affect the runtime and space usage – and so is a major topic of this module

Terminology of Rooted Trees (1)

Tree consists of nodes

- Unique root “A” with no parent
- Edges are relations $\text{parent}(A,B)$, etc
- Non-root nodes have precisely one parent
 - $\text{Parent}(F,H)$ is forbidden
- Nodes may have any number of children
 - Often regard the children as ordered left to right
 - Nodes with no children are called “leaf” nodes.
 - Otherwise, they are “internal”



Trees: Uses

Physical:

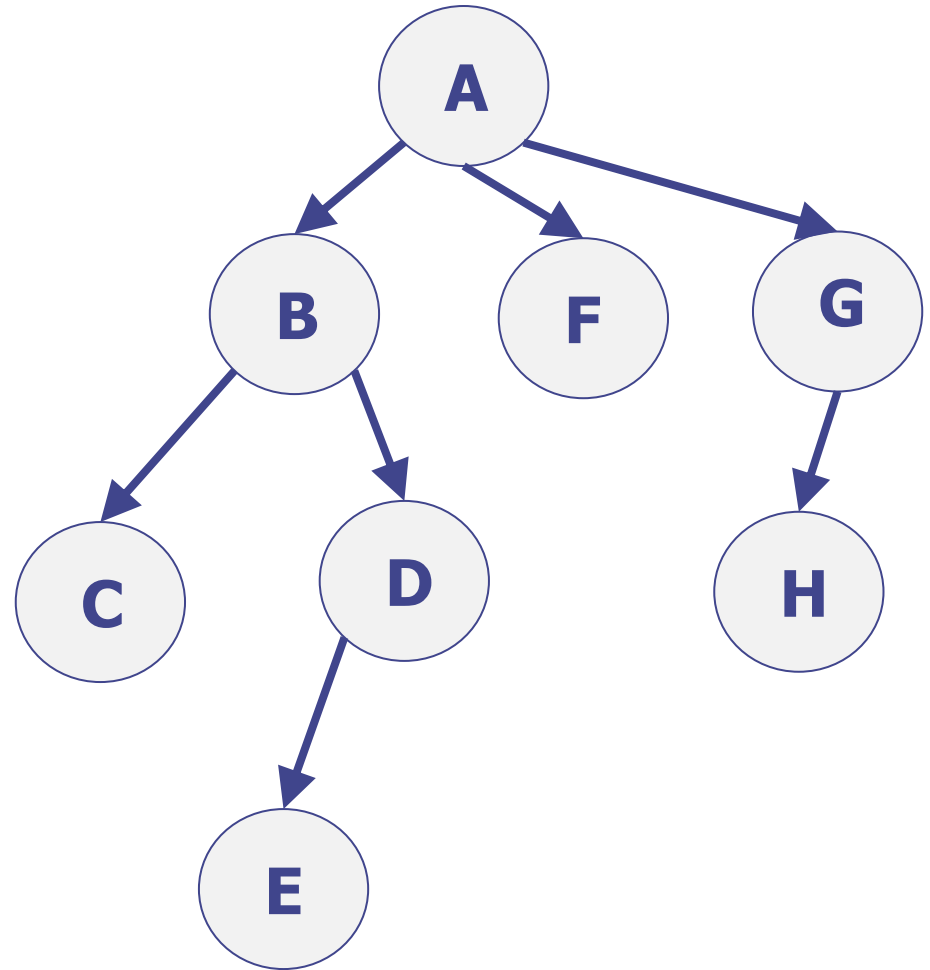
- File Systems – folders/directories Unix “/” is the root
- Organisation Hierarchies
- Syntax – parsed expression trees
 - $(* 3 (+ 4 5))$
- Data structures:
 - Heaps
 - Binary search trees

Conceptual structures in algorithms:

- Search trees in min-max, etc.
- Not “physical” but for organising ideas

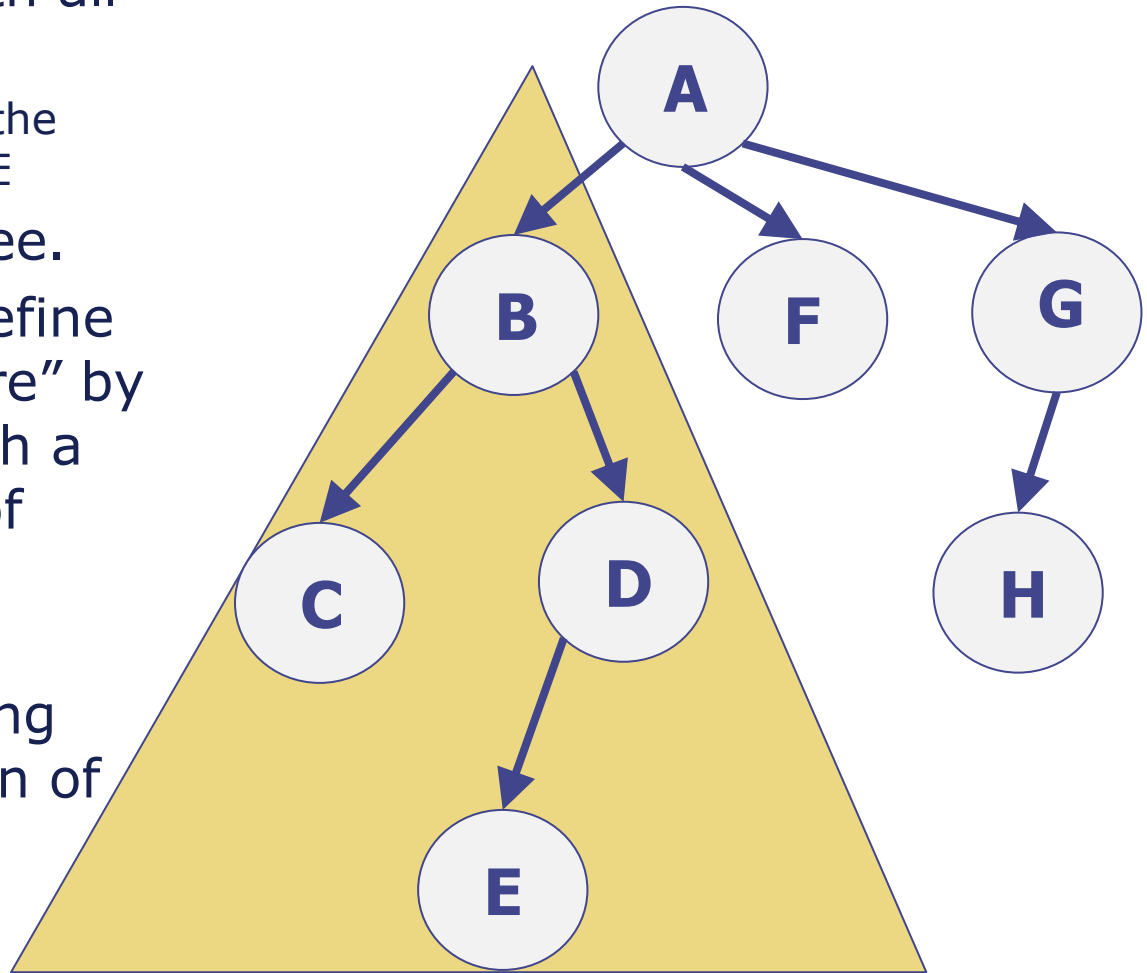
Terminology of Rooted Trees (2)

- **Descendants** of a node are the children, children of the children, etc. Any node reachable by descending
 - E.g. Descendants of B are C,D,E
- **Ancestors** of a node are the parent, parent of parent, etc. Any node reachable by ascending
 - Ancestors of D are B,A



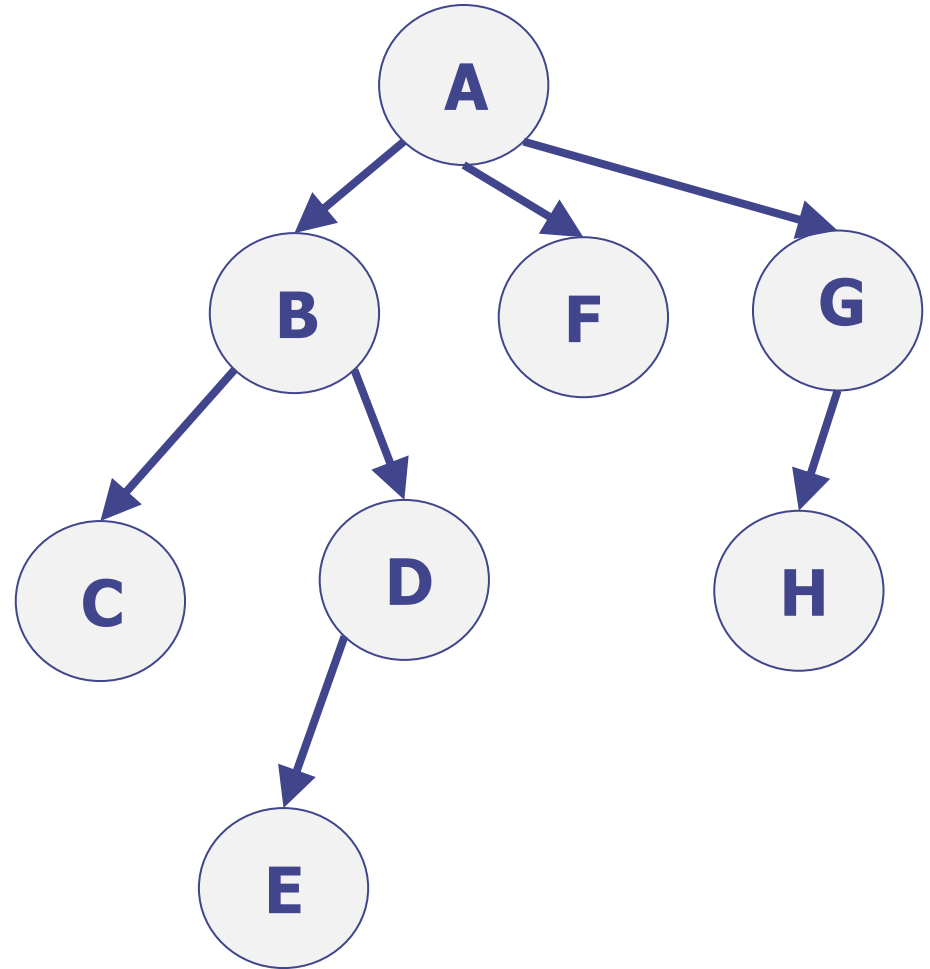
Terminology of Rooted Trees (3)

- **Sub-tree** from a node is that node together with all its descendants
 - Sub-tree of B has B as the root of a tree with C,D,E
- A sub-tree is also a tree.
- One can recursively define the “tree data structure” by associating a node with a list (possibly empty) of sub-trees
- Many algorithms work recursively by exploiting this recursive definition of the data structure



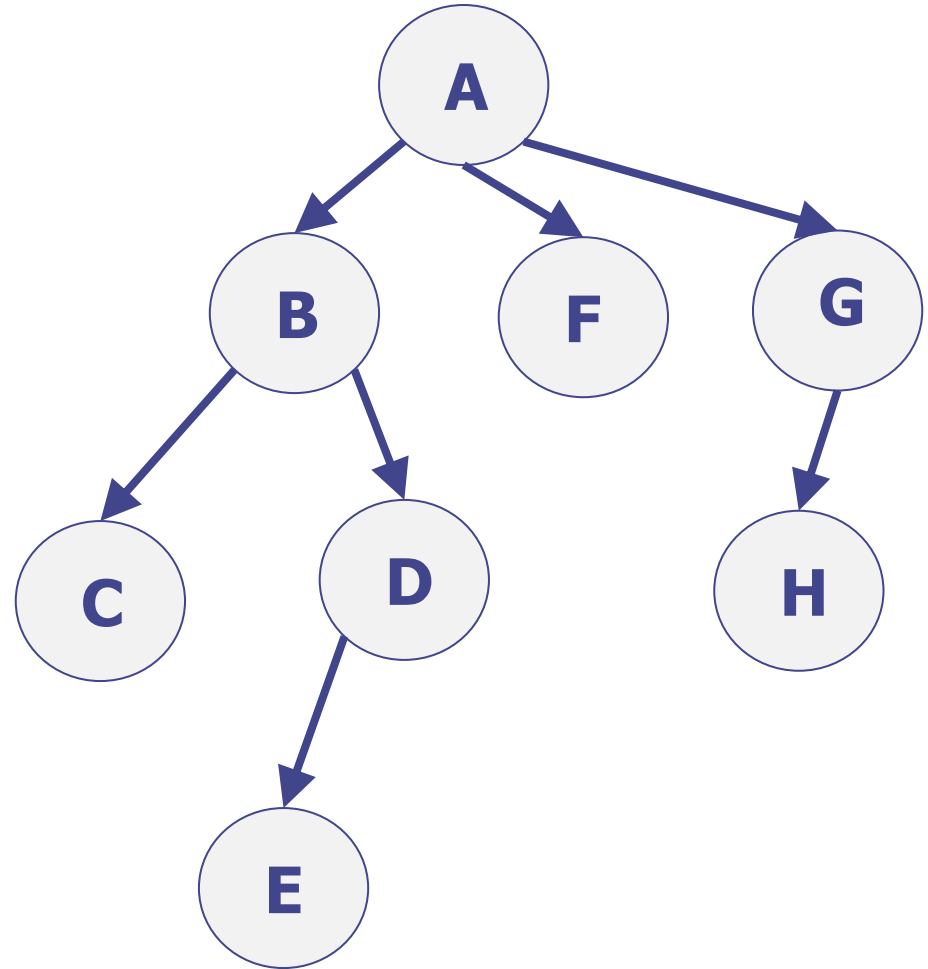
Trees: Depth and Height

- Depth of a node – number of edges that traverse from the root to the node
 - Depth of D is 2
- Height of a tree – depth of the deepest node
 - Deepest node is E
 - Height is 3
- From the root to any node there is a unique path
- Length of the longest path is the height
- **The height is an upper bound on the length of any path from root to a node**



Trees: "Arity"

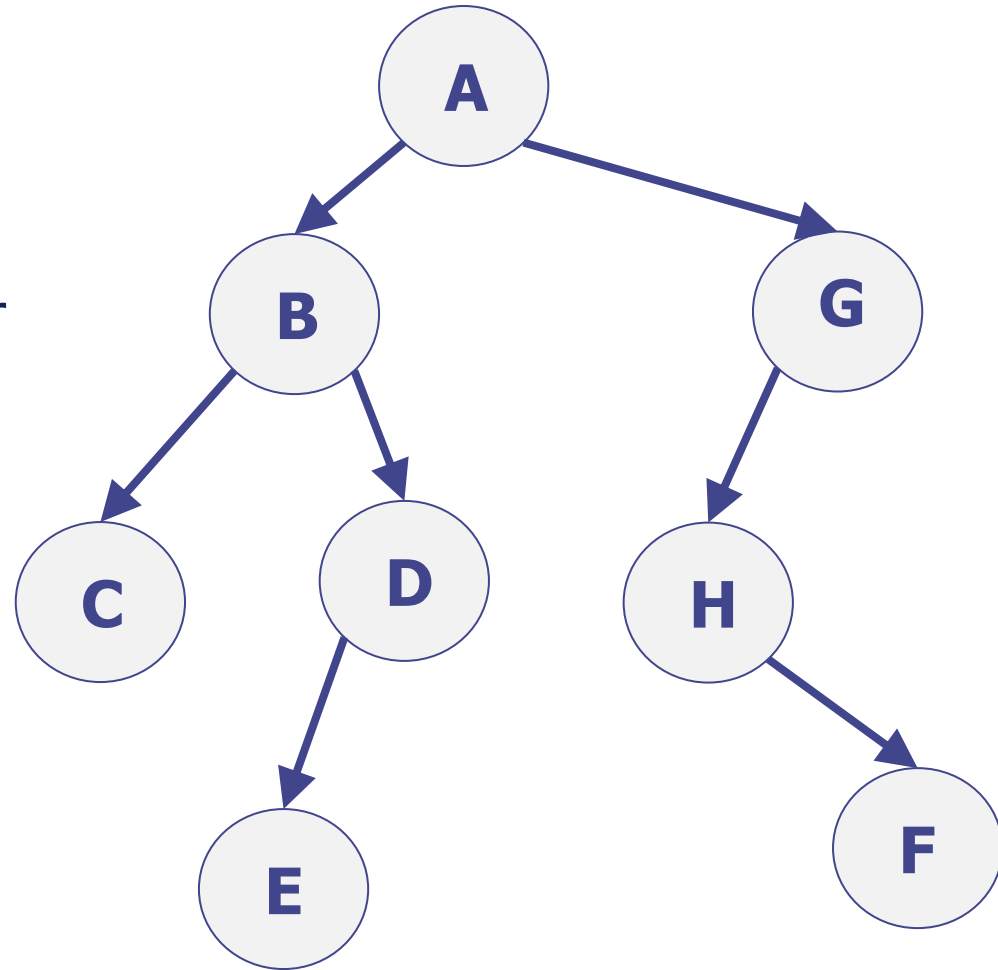
- The "Arity" of a tree is the maximum number of children that a node is allowed to have.
- It is allowed to have fewer children than the arity
- The example has arity=3
- We will often deal with "binary trees" of arity=2
- In a binary tree the children are usually called the "left" and "right"



Trees: “Binary”

- We will often deal with “binary trees” of arity=2
- In a binary tree the children are usually called the “left” and “right”
- It is allowed and common for a node to have no left child but to have a right child
 - E.g. H has a right child, F, but has no left child

Main usages in this module:
“Binary heaps”
“Binary Search trees”



Binary Trees: “Proper”

- A “proper” binary tree is one in which all the nodes have either no children or have two children
- It is not allowed for any node to have just one child

Example:

“Arithmetic Expression trees”
using binary operators

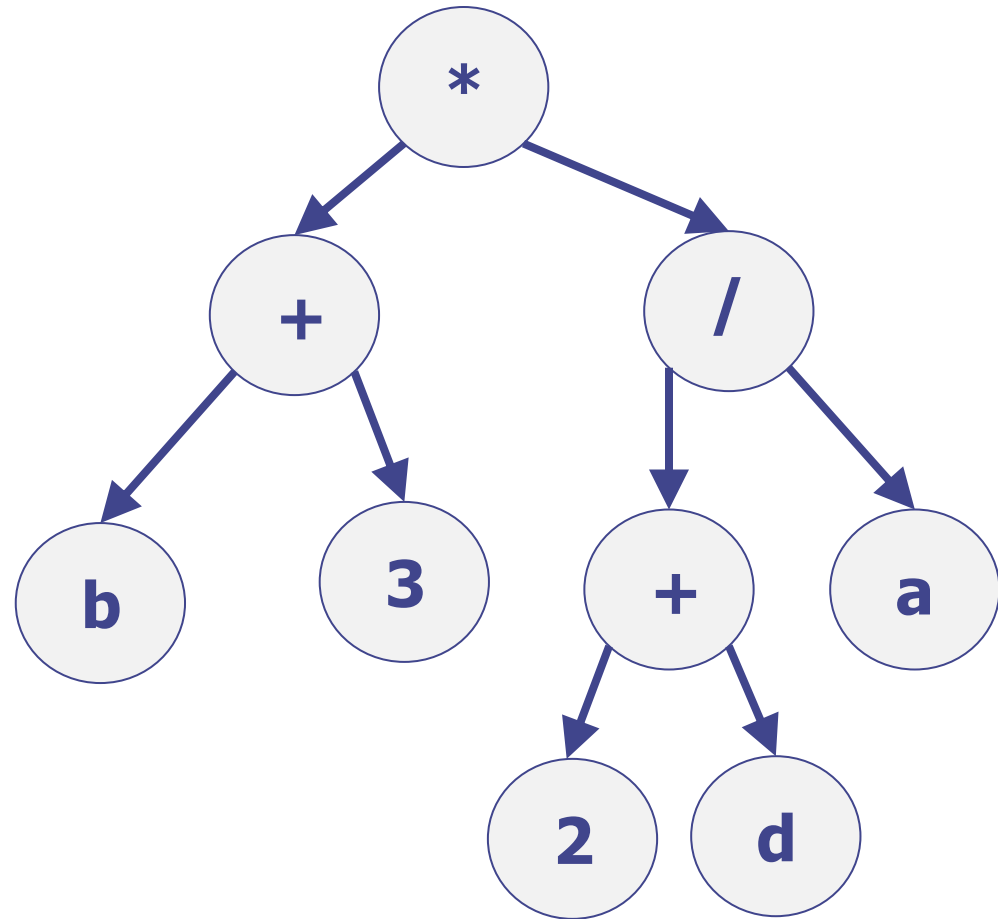
The example represents

$(* (+ b 3) (/ (+ 2 d) a))$

usually written with “infix”
operators as

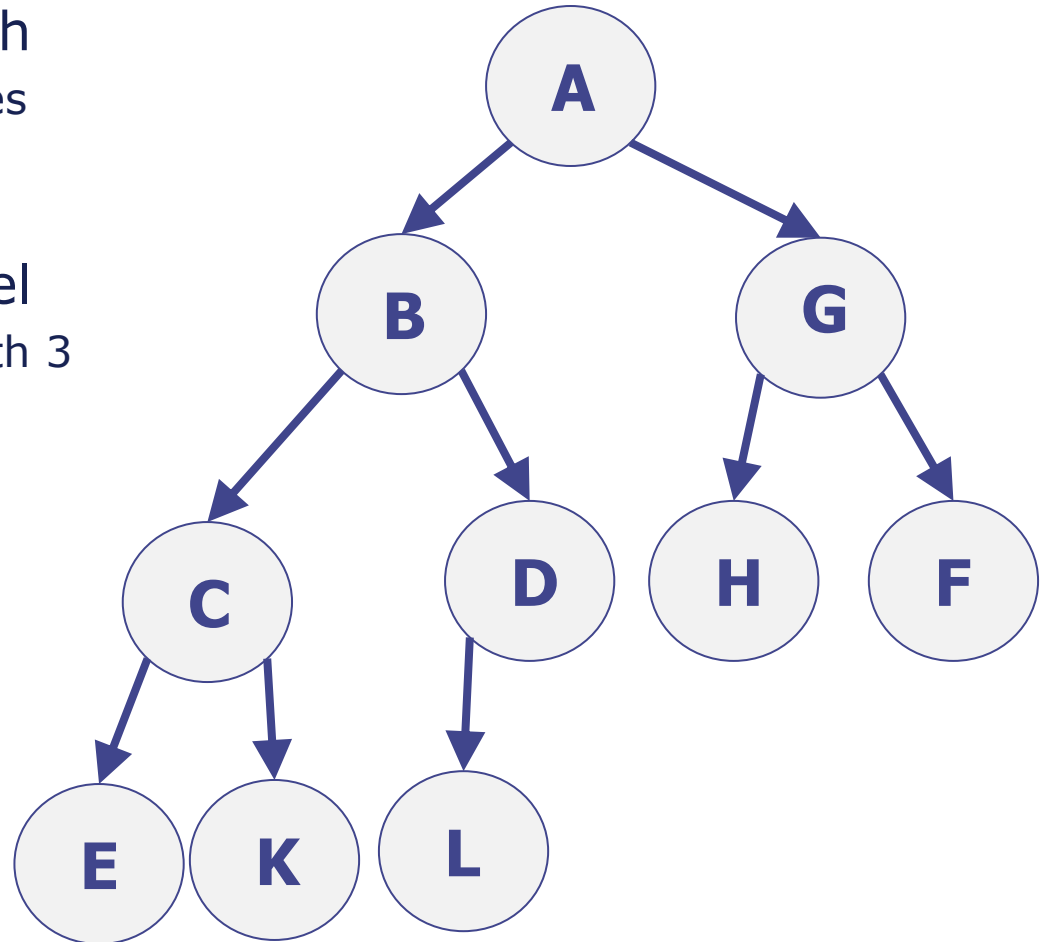
$(b+3)((2+d)/a)$

A compiler may convert this
infix expression to a tree



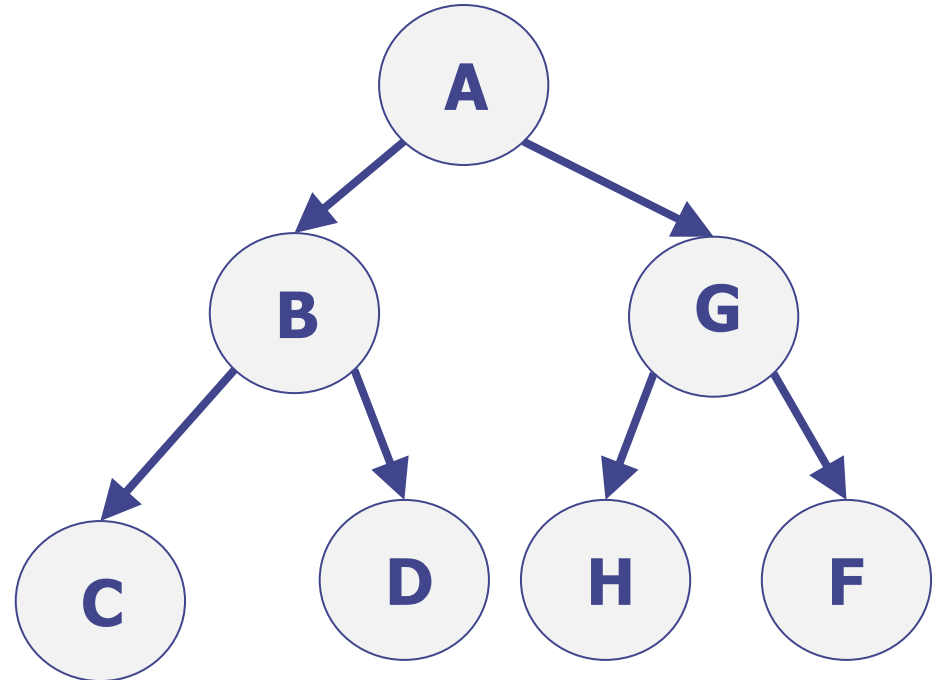
Trees: “Levels”

- A “**level**” refers to the set of nodes of a specified depth
 - E.g. level 2 comprises nodes C,D,H,F
- The “**last level**” usually refers to the deepest level
 - E.g. the last level is at depth 3 and comprises E,K,L



Trees: “Perfect”

- A tree is “perfect” if every leaf node has the same depth
- Every level is full – cannot have any more nodes
- The depth of every leaf node is the same as the height of the tree
- The example here is a perfect tree of height=2



Main usage in this module:

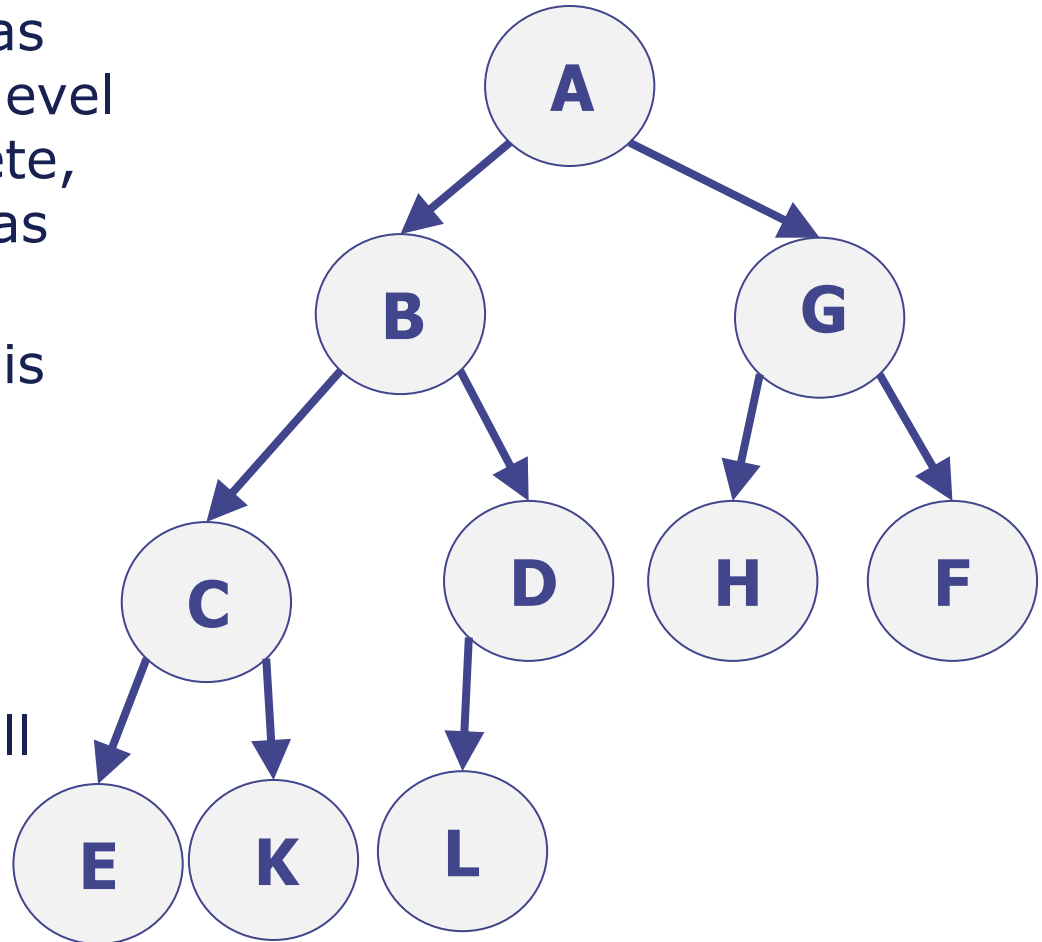
“Conceptual trees” when analysing merge sort

Trees: “Complete”

- A tree is “**complete**” if every level, except the last, is as full as possible. The last level is allowed to be incomplete, but every node must be as far left as possible
- In the example, the tree is complete as
 - Level 0, {A} is full
 - Level 1, {B,G} is full
 - Level 2, {C,D,H,F} is full
 - Level 3, {E,K,L} is not full but is fully left shifted

The rightmost node of the last level, “L”, is called the “**last node**”

- Usage: Heaps



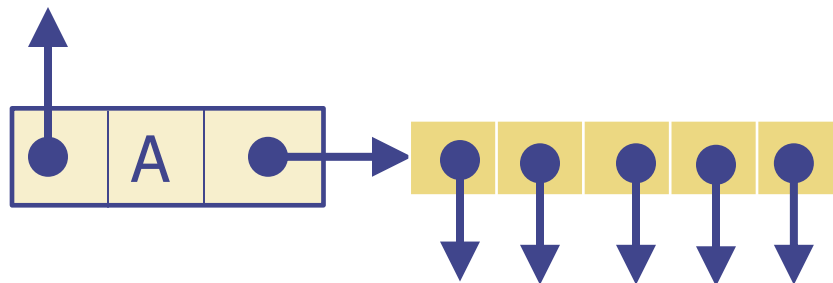
Implementation: "Nodes"

This implementation is very similar to the idea of linking nodes in a linked list – but more complicated because each node will link to its children and also (usually) to its parent.

Nodes will have a pointer to the parent.

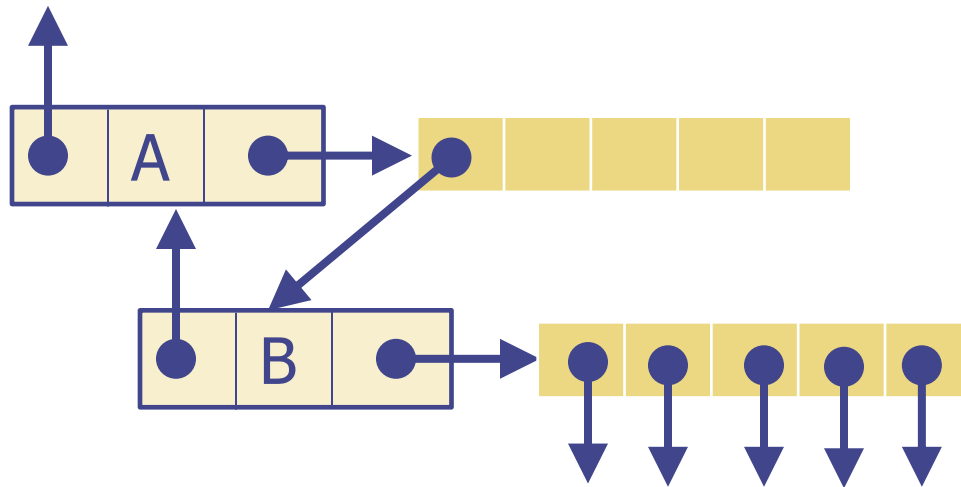
Also contains data, in this case "A", but more generally a pointer to some object

For general arity, we will need to store an array/list of pointers to the children. E.g. for a Node with 5 children we can have as below:



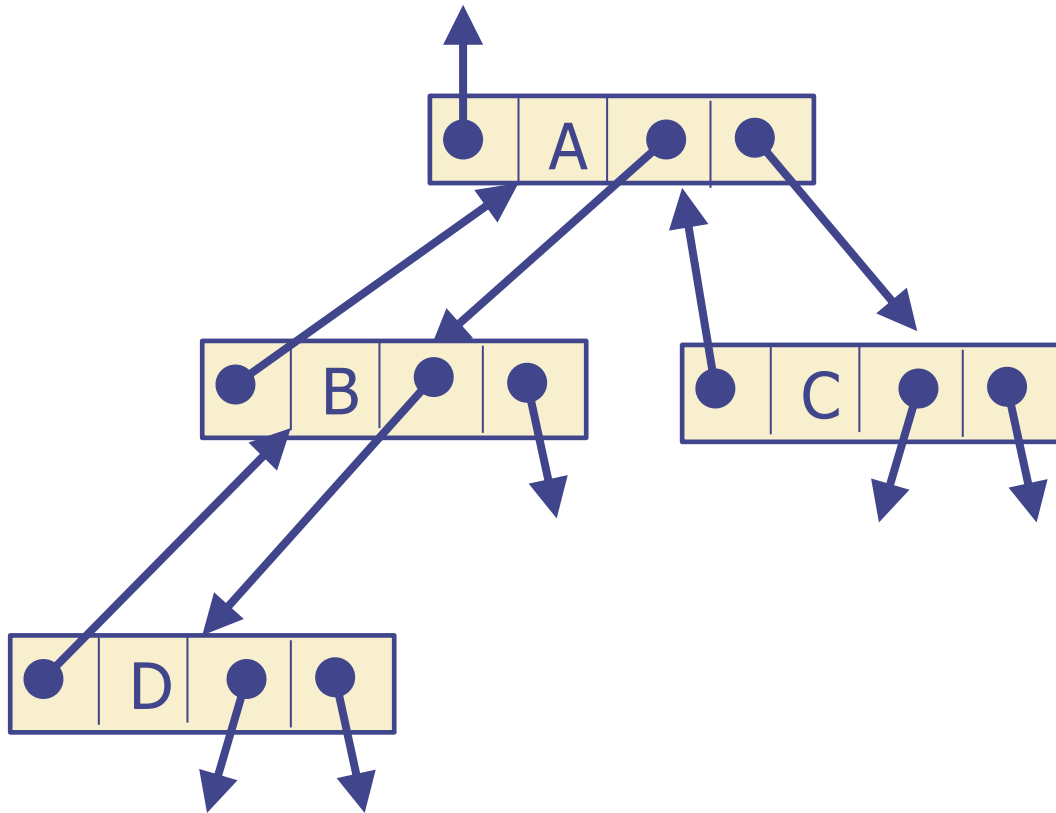
Implementation: “Linked nodes”

- We will need to stitch the nodes together and so form a tree structure – a fragment is shown below
- This is straightforward but very messy.
- Also, very prone to bugs as easy to have pointer errors.
- Good programming and software-engineering task of thinking how to organise this!



Binary tree: “Linked nodes”

- Structure is somewhat simpler if we restrict to a binary tree as the node itself can store the links to the left and right children:



Expectations

Know the terminology.

Be able (at least, in principle) to implement trees using a “linked nodes” structure